

Zcash Walkthrough

m@rek.onl

One order

Why nodes must agree on an order

Two transactions spend the same coin. Each is valid alone; together, one must lose.



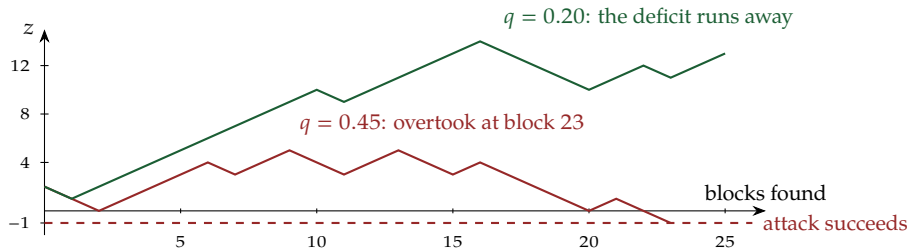
Consensus supplies a **canonical order**: the chain with the most work. Every later rule, the note tree, the nullifier set, the pool balances, is a function of that order.

The race

An attacker mines a secret branch; z is the honest chain's lead in blocks.

$$z_{i+1} = \begin{cases} z_i + 1 & \text{honest block, probability } p \\ z_i - 1 & \text{attacker block, probability } q = 1 - p \end{cases}$$

The attack succeeds when z reaches -1 .



Deriving the catch-up probability

Let $h_N(z)$ be the probability that the walk from z reaches -1 before a ceiling N .
Condition on the next block:

$$h_N(z) = p h_N(z+1) + q h_N(z-1), \quad h_N(-1) = 1, \quad h_N(N) = 0.$$

The trial solution $h_N(z) = t^z$ works exactly at the roots of

$$p t^2 - t + q = p (t-1) (t - q/p),$$

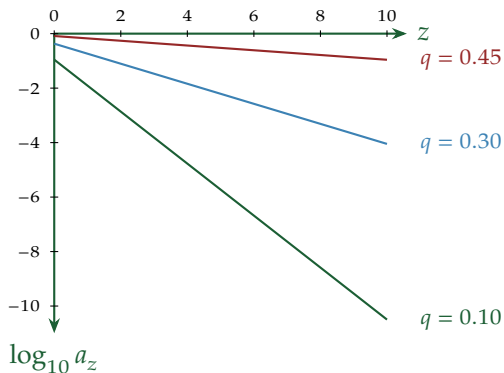
so for $q \neq p$, $h_N(z) = A + B (q/p)^z$, and the two boundaries give

$$h_N(z) = \frac{(q/p)^z - (q/p)^N}{(q/p)^{-1} - (q/p)^N} \xrightarrow{N \rightarrow \infty} a_z = (q/p)^{z+1} \quad (q < p).$$

For $q \geq p$ the limit is 1: an attacker with half the hashrate or more always catches up.

The catch-up probability

$$a_z = \Pr[\text{the walk ever reaches } -1 \text{ from } z] = \begin{cases} (q/p)^{z+1} & q < p \\ 1 & q \geq p \end{cases}$$



q	a_0	a_5
0.10	0.111	1.9×10^{-6}
0.30	0.429	6.2×10^{-3}
0.45	0.818	0.30

Each block of lead multiplies a_z by q/p : the order of old blocks becomes final exponentially fast.

What the order gives the shielded protocol

object	a function of the ordered chain
note commitment tree anchor	every cmx, appended in order the tree's root after a block
nullifier set	every nf published so far
pool balance	minus the sum of every v_{balance}

From here on: one canonical chain. The rest of the talk follows one payment through it.

Primitives

Groups and generators

curve	equation	group order
Pallas $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$	$y^2 = x^3 + 5$ over $\mathbb{F}_{p_{\text{Pallas}}}$	p_{Vesta}
Vesta	$y^2 = x^3 + 5$ over $\mathbb{F}_{p_{\text{Vesta}}}$	p_{Pallas}

Two primes $p_{\text{Pallas}}, p_{\text{Vesta}} \approx 2^{254}$: each curve's scalar field is the other's base field.
Discrete logarithms cost about 2^{126} group operations.

$$[a]P = P + \dots + P, \quad \text{Extract}_{\mathbb{P}}((x, y)) := x, \quad \text{GroupHash}(D, M) \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$$

Every generator is GroupHash of its own label: $G^{\text{SpendAuth}}$ (spend authorisation), K^{Orchard} (nullifiers), $V^{\text{Orchard}}, R^{\text{Orchard}}$ (values), $\mathbf{g}_d$ (addresses), $Q, S(j)$ (Sinsemilla). No relation among them is known, and nothing is set up in trust.

Pseudorandom functions

For a secret uniform key, outputs look uniform and independent: one secret expands into many keys.

function	built from	used for
$\text{PRF}_k(t, \dots)$	BLAKE2b, into $\mathbb{F}_{p_{\text{Pallas}}}$ or $\mathbb{F}_{p_{\text{Vesta}}}$ if needed	every derived key
$\text{PRF}_{\text{nk}}^{\text{nf}}(\rho)$	Poseidon, over $\mathbb{F}_{p_{\text{Pallas}}}$	nullifiers
$\text{KDF}(s, \text{epk})$	BLAKE2b	encryption keys

The tag t names the output: $\text{PRF}_{\text{sk}}(\text{ask})$ says nothing about $\text{PRF}_{\text{sk}}(\text{nk})$. The nullifier PRF is $\text{PRF}_{\text{nk}}^{\text{nf}}(\rho) := \text{PoseidonHash}(\text{nk}, \rho)$, an algebraic hash the proof can evaluate cheaply.

The Sinsemilla hash

Generators $S(0), \dots, S(1023)$ and a base $Q(D)$ for each domain D . Split M into 10-bit chunks $m_1, \dots, m_n$:

$$\text{Acc}_0 := Q(D), \quad \text{Acc}_i := (\text{Acc}_{i-1} + S(m_i)) + \text{Acc}_{i-1}$$

$$\text{SinsemillaHashToPoint}(D, M) := \text{Acc}_n = [2^n] Q(D) + \sum_{i=1}^n [2^{n-i}] S(m_i)$$

$$\text{SinsemillaHash}(D, M) := \text{Extract}_{\mathbb{P}}(\text{Acc}_n)$$

- A Pedersen-style hash: two messages of one length with one output give a discrete-log relation among $Q(D), S(0), \dots, S(1023)$.
- Each 10-bit step is one table lookup and two additions: cheap to prove.

Sinsemilla commitments

$$\text{SinsemillaCommit}_r(D, M) := \text{SinsemillaHashToPoint}(D, M) + [r] H_D$$

$$\text{ShortCommit}_r(D, M) := \text{Extract}_{\mathbb{P}}(\text{SinsemillaCommit}_r(D, M))$$

$r \in \mathbb{F}_{p_{\text{Vesta}}}$ uniform; H_D another hashed generator.

- **Hiding:** for uniform r the commitment is a uniform point, whatever M .
- **Binding:** two openings of one commitment give a discrete-log relation.

instance	form	input
$\text{NoteCommit}_{\text{rcm}}$	commitment	a note
$\text{Commit}_{\text{r}_{\text{ivk}}}^{\text{ivk}}$	short commitment	(ak, nk)
MerkleCRH	hash	two tree nodes

Value commitments

$$\text{cv}(v, \text{rcv}) := [v] V^{\text{Orchard}} + [\text{rcv}] R^{\text{Orchard}}$$

v a signed amount; $\text{rcv} \in \mathbb{F}_{p_{\text{Vesta}}}$ uniform.

- **Hiding:** uniform rcv gives a uniform point.
- **Binding:** one point opened to two amounts gives $\log_{R^{\text{Orchard}}} V^{\text{Orchard}}$.
- **Homomorphic:**

$$\text{cv}(v_1, r_1) + \text{cv}(v_2, r_2) = \text{cv}(v_1 + v_2, r_1 + r_2)$$

The commitments of a bundle add up to a commitment to its net value.

Key agreement and encryption

The recipient publishes (g_d, pk_d) with $pk_d = [ivk] g_d$. The sender picks a fresh esk :

$$epk := [esk] g_d, \quad s := [esk] pk_d = [esk \cdot ivk] g_d = [ivk] epk$$

$$K := \text{KDF}(s, epk), \quad C := \text{Enc}_K(\text{plaintext})$$

Enc is authenticated encryption (ChaCha20-Poly1305): decryption under a wrong key fails.

Diffie–Hellman on a base chosen by the recipient: s needs ivk or esk . The pair (epk, C) names no one.

Key derivation

A derived key is a hash of a secret and a context, under a label that names its purpose:

$$K := \text{KDF}(s, \text{epk}) = H_{\text{KDF}}(s \parallel \text{epk})$$

$$\text{ock} := \text{KDF}_{\text{ovk}}(\text{cv}^{\text{net}}, \text{cmx}, \text{epk}) = H_{\text{ock}}(\text{ovk} \parallel \text{cv}^{\text{net}} \parallel \text{cmx} \parallel \text{epk})$$

- **Extraction:** the secret s is a uniform point, not uniform bits; the hash turns it into a uniform cipher key.
- **Context:** the public transcript enters the hash, epk for K and $\text{cv}^{\text{net}}, \text{cmx}, \text{epk}$ for ock , so a key belongs to one key agreement or one Action.
- **Separation:** distinct labels make K , ock and the key-tree outputs $\text{PRF}_k(t)$ independent, even from equal inputs.

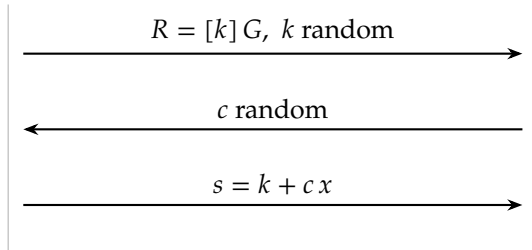
Modelled as random oracles: without the secret input, a derived key is uniform and independent of every other.

A sigma protocol

The prover knows x with $X = [x] G$.

Prover

Verifier



Verifier checks $[s] G \stackrel{?}{=} R + [c] X$

$$x, k, c, s \in \mathbb{F}_{p_{\text{Vesta}}}; G, X, R \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}).$$

Why it proves, and why it hides

Special soundness. Two accepting transcripts with the same R :

$$(R, c, s), (R, c', s') \implies x = \frac{s - s'}{c - c'}$$

whoever answers two challenges knows x .

Zero knowledge. A simulator with no x picks c and s first:

$$R := [s] G - [c] X$$

the transcript has exactly the real distribution. The verifier learns nothing it could not have made itself.

Fiat–Shamir: the verifier becomes a hash

$$c := H^{\odot}(R, X, M)$$

- The prover cannot pick R after c : c depends on R .
- No interaction: anyone can check (R, s) later.
- Binding a message M into c makes (R, s) a signature on M .
- **sigma protocol + Fiat–Shamir = Schnorr signature.**

$H^{\odot}$ is modelled as a random oracle; Halo 2 draws its challenges the same way.

Schnorr signatures

Key $x \in \mathbb{F}_{p_{\text{Vesta}}}$, public $X := [x] B$; to sign M , pick a fresh k :

$$R := [k] B, \quad c := H^{\otimes}(R, X, M), \quad s := k + c x$$

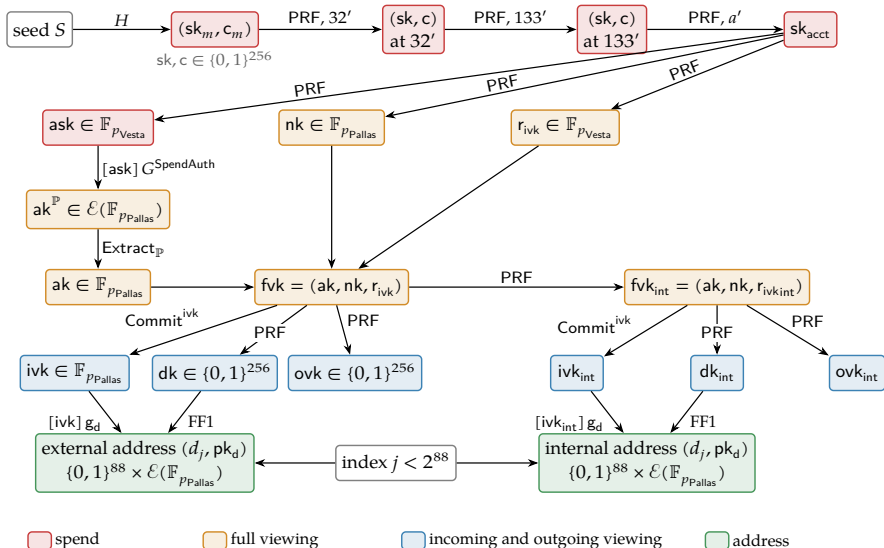
$$\text{verify}(R, s): \quad [s] B = R + [c] X$$

- Re-randomisable: $(x + \alpha, X + [\alpha] B)$ is a fresh-looking key pair, and signing under it still needs x .
- RedPallas: spend authorisation on $B = G^{\text{SpendAuth}}$ with re-randomised keys; the binding signature on $B = R^{\text{Orchard}}$.

Unforgeable under discrete logarithms, with $H^{\otimes}$ a random oracle.

Keys

The key structure



From the seed to an account key

$$(\text{sk}_m, c_m) := H(S), \quad (\text{sk}_i, c_i) := \text{PRF}_{c_{\text{par}}}(\text{sk}_{\text{par}}, i)$$

S the wallet seed; H a hash with two halves; each chain code c keys the next step.

Path $m/32'/133'/a'$: purpose, coin type, account.

- Hardened only: a child reveals nothing about its parent or its siblings.
- Account a keeps $\text{sk} := \text{sk}_{m/32'/133'/a'}$ and drops the chain code.

Spending secrets

$$\text{ask} := \text{PRF}_{\text{sk}}(\text{ask}) \in \mathbb{F}_{p_{\text{Vesta}}}$$

$$\text{nk} := \text{PRF}_{\text{sk}}(\text{nk}) \in \mathbb{F}_{p_{\text{Pallas}}}$$

$$r_{\text{ivk}} := \text{PRF}_{\text{sk}}(r_{\text{ivk}}) \in \mathbb{F}_{p_{\text{Vesta}}}$$

$$\text{ak}^{\mathbb{P}} := [\text{ask}] G^{\text{SpendAuth}}, \quad \text{ak} := \text{Extract}_{\mathbb{P}}(\text{ak}^{\mathbb{P}})$$

The key ask signs spends; nk keys nullifiers; r_{ivk} blinds the viewing key.

Viewing keys

$$\text{ivk} := \text{Commit}_{r_{\text{ivk}}}^{\text{ivk}}(\text{ak}, \text{nk}), \quad (\text{dk}, \text{ovk}) := \text{PRF}_{r_{\text{ivk}}}(\text{ak}, \text{nk})$$

key	lets its holder
ask	spend
$(\text{ak}, \text{nk}, r_{\text{ivk}})$	see spends (nullifiers)
(dk, ivk)	see incoming notes, make addresses
ovk	recover what it sent

The address

For an index j :

$$d := \text{FF1}_{\text{dk}}(j), \quad g_d := \text{GroupHash}(d), \quad \text{pk}_d := [\text{ivk}] g_d$$

The address is (d, pk_d) .

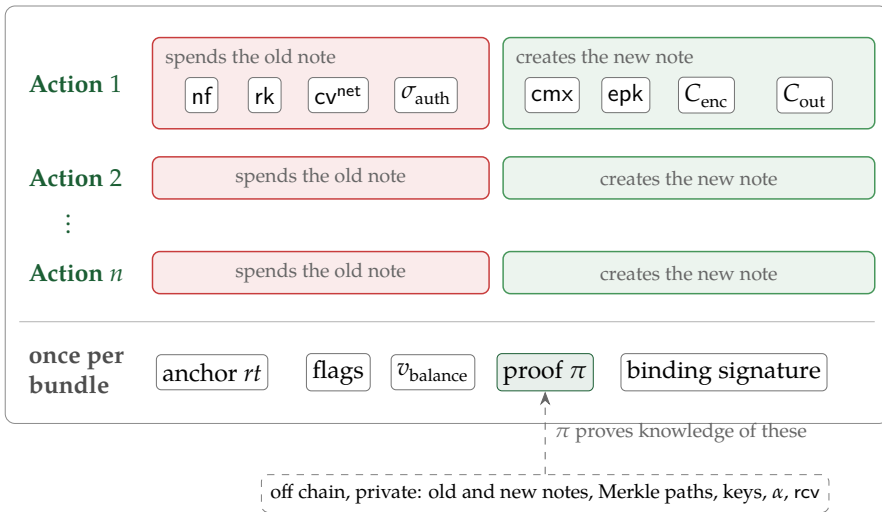
FF1 is a keyed pseudorandom permutation of 88-bit strings.

- 2^{88} addresses per key, all with $\text{pk}_d = [\text{ivk}] g_d$.
- Two addresses of one key are unlinkable without dk or ivk.
- The address is all a sender needs.

Someone pays you

The bundle

bundle, on chain



The Action



	published	what it is
old note	nf	its nullifier
	rk	a re-randomised spend authorisation key
	cv^{net}	a commitment to old minus new value
	σ_{auth}	the spend authorisation signature, checked against rk
new note	cmx	its commitment
	epk, C_{enc}	the note, encrypted to its recipient
	C_{out}	(pk_d, esk) under the sender's ovk: reopens C_{enc}

Either note may be a dummy of value 0. No field names a sender, a recipient or an amount.

Trial decryption

For every Action on the chain, the wallet tries its ivk:

1. $s := [\text{ivk}] \text{epk}$, $K := \text{KDF}(s, \text{epk})$, $(d, v, \text{rseed}, \text{memo}) := \text{Dec}_K(C_{\text{enc}})$;
2. $\rho := \text{nf}$ of the same Action; esk , ψ from rseed and ρ ;
3. $g_d := \text{GroupHash}(d)$, $\text{pk}_d := [\text{ivk}] g_d$, rcm from rseed and $(g_d, \text{pk}_d, v, \rho, \psi)$;
4. accept only if $[\text{esk}] g_d = \text{epk}$ and
 $\text{Extract}_{\mathbb{P}}(\text{NoteCommit}_{\text{rcm}}(g_d, \text{pk}_d, v, \rho, \psi)) = \text{cmx}$.

An Action meant for someone else fails the decryption or step 4. An accepted note is the unique opening of cmx : the sender cannot hand over a note the wallet could not later spend.

Light wallets run this on a compact form of each Action: nf , cmx , epk and the start of C_{enc} , without the memo.

The note

Decryption yields

$$n = ((d, \text{pk}_d), v, \rho, \psi, \text{rcm})$$

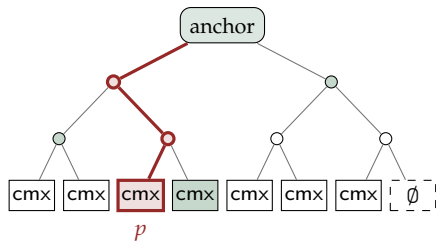
with v the amount and ρ the nullifier of the note spent in the same Action. One seed fixes the rest:

$$\begin{aligned} \text{esk} &:= \text{PRF}_{\text{rseed}}(\text{esk}, \rho), & \psi &:= \text{PRF}_{\text{rseed}}(\text{psi}, \rho), \\ \text{rcm} &:= \text{PRF}_{\text{rseed}}(\text{rcm}, g_d, \text{pk}_d, v, \rho, \psi) \end{aligned}$$

$$\text{cm} := \text{NoteCommit}_{\text{rcm}}(g_d, \text{pk}_d, v, \rho, \psi), \quad \text{cmx} := \text{Extract}_{\mathbb{P}}(\text{cm})$$

The chain holds cmx in the note commitment tree; the note itself never appears.

Keeping a witness



The note's cmx sits at position pos of a depth-32 tree. With $h_t := \text{MerkleCRH}(31 - t, \cdot, \cdot)$:

$$a_0 := \text{cmx}, \quad a_{t+1} := \begin{cases} h_t(a_t, s_t) & b_t = 0 \\ h_t(s_t, a_t) & b_t = 1 \end{cases}$$

Valid path: $a_{32} = rt$, the anchor.

Every sibling s_t is a function of the cmx values appended so far: the wallet updates the path from the blocks it scans and never reveals pos .

Watching for spends

The wallet computes its note's nullifier once:

$$\text{nf} := \text{Extract}_{\mathbb{P}}([(\text{PRF}_{\text{nk}}^{\text{nf}}(\rho) + \psi) \bmod p_{\text{Pallas}}] K^{\text{Orchard}} + \text{cm})$$

and marks the note spent when nf appears in a scanned block.

- Deterministic: one note, one nullifier.
- Unlinkable: without nk it looks random.
- The wallet now holds everything a spend needs: the note, pos, the path, an anchor.

Spending

Spending the old note

$$\begin{array}{ll} \text{nf}^{\text{old}} := \text{DeriveNullifier}_{\text{nk}}(\rho, \psi, \text{cm}) & \text{published} \\ \text{rsk} := \text{ask} + \alpha, \quad \text{rk} := \text{ak}^{\mathbb{P}} + [\alpha] G^{\text{SpendAuth}} & \text{fresh } \alpha \text{ per Action} \\ \text{cv}^{\text{net}} := \text{cv}(v^{\text{old}} - v^{\text{new}}, \text{rcv}) & \text{fresh rcv} \end{array}$$

- rk is a uniform point: two spends by one key are unlinkable.
- cv^{net} is a uniform point: the amount is hidden, and commitments add up.

Creating the new note

Each output is a new note, made as the received one was, with a fresh rseed and

$$\rho^{\text{new}} := \text{nf}^{\text{old}}.$$

It is encrypted to the payee's address (d, pk_d) :

$$\begin{aligned} \text{epk} &:= [\text{esk}] g_d, & K &:= \text{KDF}([\text{esk}] \text{pk}_d, \text{epk}), & C_{\text{enc}} &:= \text{Enc}_K(d, v, \text{rseed}, \text{memo}) \\ \text{ock} &:= \text{KDF}_{\text{ovk}}(\text{cv}^{\text{net}}, \text{cmx}, \text{epk}), & C_{\text{out}} &:= \text{Enc}_{\text{ock}}(\text{pk}_d, \text{esk}) \end{aligned}$$

The payee opens C_{enc} with ivk ; the payer opens C_{out} with ovk . Change is an output to an internal address of the payer.

A payment of 5 ZEC

A note of 7 ZEC pays 5 ZEC; every Action has one spend and one output:

Action	v^{old}	v^{new}	$v^{\text{old}} - v^{\text{new}}$
note $\rightarrow$ payee	7 ZEC	5 ZEC	2 ZEC
dummy $\rightarrow$ change	0	1.9999 ZEC	-1.9999 ZEC
v_{balance} (the fee)			0.0001 ZEC

The value balance v_{balance} leaves the shielded pool: here, as the fee.

What the proof asserts

For public $(rt, cv^{\text{net}}, nf^{\text{old}}, rk, cmx, \text{flags})$, the prover knows a witness such that:

1. the old note opens cm^{old} , and cm^{old} has a valid path to rt (if $v^{\text{old}} \neq 0$);
2. $nf^{\text{old}} = \text{DeriveNullifier}_{nk}(\rho, \psi, cm^{\text{old}})$;
3. $rk = ak^{\mathbb{P}} + [\alpha] G^{\text{SpendAuth}}$, and $pk_d^{\text{old}} = [\text{Commit}_{r_{ivk}}^{\text{ivk}}(ak, nk)] g_d^{\text{old}}$;
4. $cv^{\text{net}} = cv(v^{\text{old}} - v^{\text{new}}, rcv)$;
5. the new note opens cmx , with $\rho^{\text{new}} = nf^{\text{old}}$;
6. the flags hold (spends, outputs, cross-address).

One Halo 2 proof for all Actions of the bundle.

Signatures

Every signature signs SigHash, a digest of the transaction without its proofs, signatures and anchor.

- **Spend authorisation**, per Action: under rsk , checked against rk . Only a holder of ask can produce it.
- **Binding**, per bundle:

$$bsk := \sum_i rcv_i, \quad bvk := \sum_i cv_i - [v_{\text{balance}}] V^{\text{Orchard}}$$

under bsk , checked against bvk .

Verification

What a node checks

check	guarantees
the proof, on the public fields	statement 1–6 for every Action
rt is a past anchor	the spent note exists
$nf \notin$ nullifier set	not spent before
spend signature under rk	the owner authorised it
binding signature under bvk	value is conserved

Then: every nf joins the set, every cmx joins the tree. No step reads a note.

Why the binding signature enforces balance

Commitments add:

$$\sum_i cv_i = \left[\sum_i (v_i^{\text{old}} - v_i^{\text{new}}) \right] V^{\text{Orchard}} + \left[\sum_i rcv_i \right] R^{\text{Orchard}}$$

so

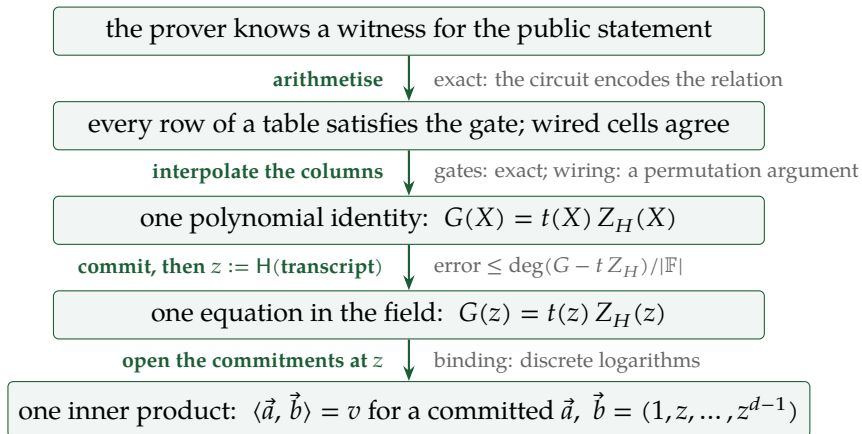
$$bvk - [bsk] R^{\text{Orchard}} = \left[\sum_i (v_i^{\text{old}} - v_i^{\text{new}}) - v_{\text{balance}} \right] V^{\text{Orchard}}.$$

A valid signature proves knowledge of b with $bvk = [b] R^{\text{Orchard}}$. If the values did not balance, b would yield $\log_{R^{\text{Orchard}}} V^{\text{Orchard}}$, a relation between two hashed generators: infeasible.

With 64-bit values and fewer than 2^{16} Actions, the sum lies in $(-2^{80}, 2^{80})$ and $2^{80} \ll p_{\text{Vesta}}$: balance modulo p_{Vesta} is balance over the integers.

How Halo 2 proves

The proving mechanism



Each step shrinks the claim. The inner-product argument folds the last line $\log_2 d$ times; the verifier recomputes the challenges and checks only the last two lines.

Zero knowledge: random blinding rows in the table and a blinder in every commitment.

The claim becomes a table

“I know x with $x^3 + x + 5 = 35$ ” over $\mathbb{F}_{97}$; the witness $x = 3$:

$$3 \xrightarrow{\text{square}} 9 \xrightarrow{\times 3} 27 \xrightarrow{+3} 30 \xrightarrow{+5} 35.$$

One operation per row, one gate equation on every row:

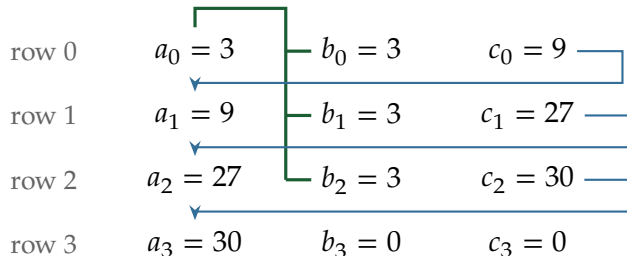
$$q_M a b + q_L a + q_R b + q_O c + q_C + \text{PI} = 0$$

every cell in $\mathbb{F}_{97}$.

row	advice			fixed					instance	
	a	b	c	q_M	q_L	q_R	q_O	q_C	PI	
0	3	3	9	1	0	0	-1	0	0	$x \cdot x = x^2$
1	9	3	27	1	0	0	-1	0	0	$x^2 \cdot x = x^3$
2	27	3	30	0	1	1	-1	0	0	$x^3 + x = s$
3	30	0	0	0	1	0	0	5	-35	$s + 5 = 35$

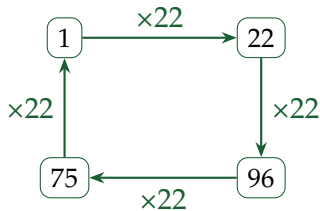
Advice: the prover's secret trace. Fixed: the circuit. Instance: the public 35. Copy constraints wire the rows; range checks are lookups into a table.

Equal values must be wired together



Equal numbers in a table are not a constraint. These *copy constraints* are enforced separately, by a permutation argument.

Give the rows multiplicative labels



$$\omega = 22, \quad \omega^2 = 96 = -1, \quad \omega^4 = 1$$

$$H = \{1, 22, 96, 75\} \subset \mathbb{F}_{97}$$

Row i gets the label ω^i : a cycle of field values.

Columns become polynomials

Row i lives at $\omega^i \in H$. A column with entries $v_0, \dots, v_{n-1}$ becomes the one polynomial of degree $< n$ through them:

$$v(\omega^i) = v_i \quad (0 \leq i < n), \quad v(X) = \sum_{i=0}^{n-1} v_i L_i(X), \quad L_i(\omega^j) = \begin{cases} 1 & j = i \\ 0 & j \neq i \end{cases}$$

Lagrange interpolation; the basis L_i depends only on H .

column	entries at 1, 22, 96, 75	polynomial in $\mathbb{F}_{97}[X]$
a	3, 9, 27, 30	$90 + 61X + 22X^2 + 24X^3$
b	3, 3, 3, 0	$75 + 32X + 25X^2 + 65X^3$
c	9, 27, 30, 0	$65 + 16X + 3X^2 + 22X^3$
q_M	1, 1, 0, 0	$49 + 19X + 30X^3$

The fixed columns q_L, q_R, q_O, q_C and PI likewise. The table is now nine polynomials of degree < 4 .

The gate becomes one polynomial

Combine the column polynomials as the gate combines the columns:

$$G(X) := q_M a b + q_L a + q_R b + q_O c + q_C + \text{PI}$$

At the point of row i it evaluates to the gate equation of row i :

$$G(\omega^i) = q_{M,i} a_i b_i + q_{L,i} a_i + q_{R,i} b_i + q_{O,i} c_i + q_{C,i} + \text{PI}_i$$

every row satisfies the gate $\iff G(\omega^i) = 0$ for all $i \iff G$ vanishes on H

$\deg G \leq 3(n-1) = 9$: the product $q_M ab$ has three factors of degree $< n$.

Toy:

$$G(X) = 36 + 27X + 54X^2 + 50X^3 + 77X^4 + 24X^5 + 43X^6 + 47X^7 + 81X^8 + 46X^9$$

with $G(1) = G(22) = G(96) = G(75) = 0$.

Vanishing is divisibility

The vanishing polynomial of H :

$$Z_H(X) := \prod_{h \in H} (X - h) = X^4 - 1$$

H is the set of fourth roots of unity, so $Z_H = X^4 - 1$.

A polynomial has a root at h exactly when $X - h$ divides it. Over the four distinct points of H ,

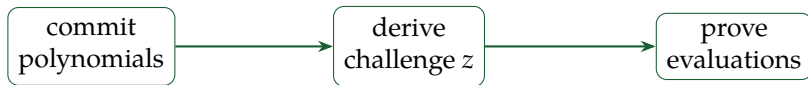
$$G \text{ vanishes on } H \iff Z_H \mid G \iff G = t \cdot Z_H \text{ for some } t, \deg t \leq 5.$$

Toy, honest: $t(X) = 61 + 70X + 43X^2 + 47X^3 + 81X^4 + 46X^5$, and $G = t \cdot Z_H$ exactly.

Toy, the lie $x^2 = 10$ in row 0: division by Z_H leaves the remainder $24(1 + X + X^2 + X^3) \neq 0$, so no quotient t exists.

Every gate holding *is* a polynomial identity. A failed gate cannot be.

One random point



1. The prover *commits* to a, b, c and t .
2. The challenge $z := H(\text{transcript})$ — Fiat–Shamir.
3. The prover opens $a(z), b(z), c(z), t(z)$.
4. The verifier checks one equation:

$$G(z) \stackrel{?}{=} t(z) Z_H(z)$$

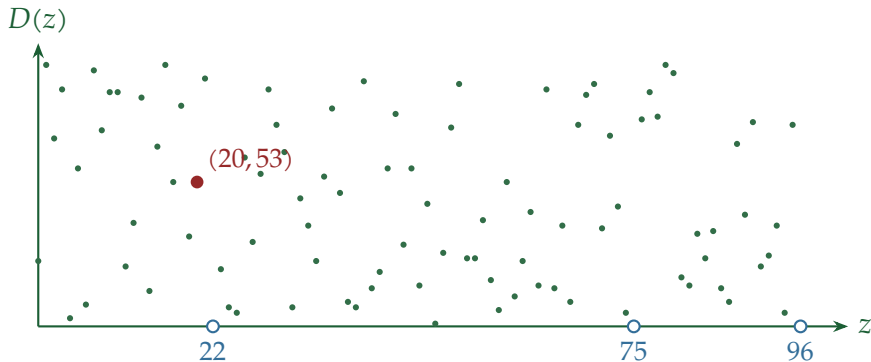
$z \in \mathbb{F}_{97}$ in the toy; $z \in \mathbb{F}_p$ in the protocol.

Honest, $z = 20$: $G(20) = 75 = 67 \cdot 46$. The row-0 lie: $G^*(20) = 20 \neq 64$.

A false identity is frozen before z is drawn; it survives only if z is one of its few roots.

A nonzero polynomial has few roots

The remainder $D(X) = 24(1 + X + X^2 + X^3)$ over $\mathbb{F}_{97}$: zeros at 22, 75, 96 only.



For uniform z this cheat passes with probability $3/97$; over the protocol's field, about 2^{-240} .

Committing to a polynomial

$$P := \langle \vec{a}, \vec{G} \rangle + [r] H, \quad \vec{a} = (a_0, \dots, a_{d-1}), \quad \vec{G} \text{ independent points}$$

$$a(z) = \langle \vec{a}, \vec{b} \rangle, \quad \vec{b} := (1, z, z^2, \dots, z^{d-1})$$

$a_i, r, z \in \mathbb{F}_p$; $\vec{G}, H, P \in \mathcal{U}$, the Vesta curve, whose scalar field is $\mathbb{F}_p$.

- A Pedersen commitment to the coefficient vector: hiding, binding.
- Opening at z is an inner product with a public vector.
- The problem: convince the verifier of $\langle \vec{a}, \vec{b} \rangle = v$ in fewer than d group elements, without sending $\vec{a}$.

The inner-product argument: one fold

Put the commitment and the claimed value into one point:

$$C := P + [v] U = \langle \vec{a}, \vec{G} \rangle + [\langle \vec{a}, \vec{b} \rangle] U$$

Split every vector in half. The prover sends two cross terms:

$$L := \langle \vec{a}_{\text{lo}}, \vec{G}_{\text{hi}} \rangle + [\langle \vec{a}_{\text{lo}}, \vec{b}_{\text{hi}} \rangle] U, \quad R := \langle \vec{a}_{\text{hi}}, \vec{G}_{\text{lo}} \rangle + [\langle \vec{a}_{\text{hi}}, \vec{b}_{\text{lo}} \rangle] U$$

The verifier sends a challenge u , and both sides fold:

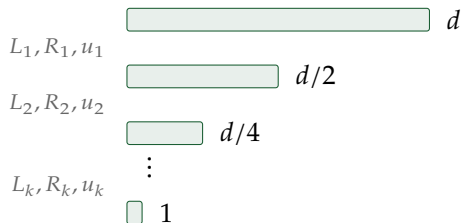
$$\vec{a}' := u \vec{a}_{\text{lo}} + u^{-1} \vec{a}_{\text{hi}}, \quad \vec{G}' := u^{-1} \vec{G}_{\text{lo}} + u \vec{G}_{\text{hi}}, \quad \vec{b}' := u^{-1} \vec{b}_{\text{lo}} + u \vec{b}_{\text{hi}}$$

$$C' := C + [u^2] L + [u^{-2}] R = \langle \vec{a}', \vec{G}' \rangle + [\langle \vec{a}', \vec{b}' \rangle] U$$

The same claim at half the length; the verifier never sees $\vec{a}$.

The blinding terms $[\cdot] H$ fold along in the same way and are omitted.

Folding down to one



After $k = \log_2 d$ folds every vector has length 1, and the prover reveals the last scalar a .

The verifier checks, with G^* and b^* its own folded $\vec{G}$ and $\vec{b}$:

$$C + \sum_{j=1}^k ([u_j^2] L_j + [u_j^{-2}] R_j) \stackrel{?}{=} [a] G^* + [a b^*] U$$

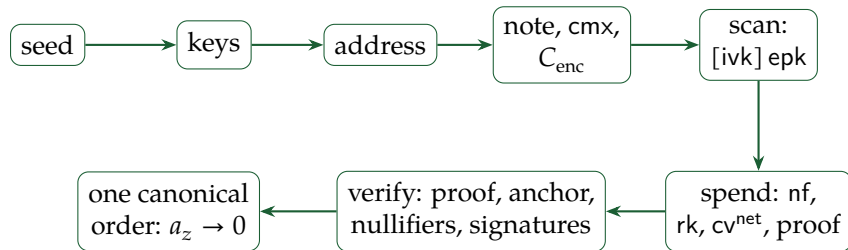
- The proof: $2 \log_2 d$ points and one scalar, in place of d scalars.
- A prover who passes with a wrong v yields a discrete-log relation among $\vec{G}$ and U .

The Orchard circuit

- Arithmetic over $\mathbb{F}_p$, so Pallas points cm, rk, cv are native; the polynomial commitments live on Vesta.
- Gadgets: Sinsemilla (commitments, tree), Poseidon (nullifier), elliptic-curve arithmetic, lookups for range checks.
- The instance: $anchor, cv, nf, rk, cmx, flags$.
- Generators from a hash: no trusted setup, no ceremony.
- Proof size grows linearly in the number of Actions.

Every clause of the statement is a set of rows; one proof, one random point, one inner-product argument.

The whole path



The verifier learns that every check passes, the fee, the anchor and the flags, and nothing about the notes.